

目 录

择栖工作室 2026 年招新题（后端方向）	2
Part 0	2
1. 提交方式	2
2. 编程语言	2
3. 基础工具与技能	2
4. 温馨提示	2
Part 1：后端基础	3
一、编程基础	3
1. 简答题	3
2. 编程题：网站日志分析器	3
3. 提交内容	4
二、网络和 HTTP	5
1. 简答题	5
2. API 设计题：用户管理系统	5
3. 提交内容	6
三、关系型数据库与 SQL	7
1. 简答题	7
2. 编程题：练习使用 SQL 语言	7
3. 提交内容	7
Part 2：项目实战	8
四、用户管理系统	8
1. 简答题	8
2. 编程题	8
3. 提交内容	8
五、挑战题	10
1. 简答题	10
2. 扩展用户管理系统	10
3. 提交内容	10

择栖工作室 2026 年招新题（后端方向）

Part 0

1. 提交方式

- 提交内容请查看每一道大题的具体说明。
- 请为每个大题单独创建一个文件夹，命名为“题号、模块名称”，例如 一、编程基础。
- 请将所有大题文件夹存放至一个文件夹内，打包成压缩包后，发送至电子邮箱 lengshui2306@163.com，邮件中备注学号、姓名、使用的编程语言。

2. 编程语言

请从以下语言中选择一个

- Java（最推荐）
- Go
- Python

3. 基础工具与技能

请自学以下两项基础技能

- Git：一种分布式版本控制系统，程序员的基础，用于版本控制与多人协作
- Markdown：一种轻量级标记语言，在你未来的学习中会经常遇到

4. 温馨提示

- 笔试结束后，将根据提交的解答发放面试资格。
- 遇到问题可以随时在招新群内提问，或私聊学长学姐。
- 即使没有完成全部的任务也可以提交，如果你的解答足够优秀，也有机会获得面试资格。
- 不禁止使用 AI，但希望仅用于辅助学习与排查问题，请勿直接提交 AI 生成的答案。
- 截止时间为 2026 年 11 月 22 日 23:59。

Part 1: 后端基础

一、编程基础

1. 简答题

- 什么是面向对象？类与对象有什么关系？（若使用 Go，可从 struct 与方法、interface 的角度作答）

2. 编程题：网站日志分析器

题目描述：

- 你将会得到一个服务器的访问日志文件 `access.log`，每一行记录一次用户请求。
- 请编写一个程序分析该文件并生成统计报告。

每行日志格式：

```
[IP地址] - - [日期和时间] "请求方法 URL 协议" 状态码 响应大小
```

日志示例：

```
192.168.1.1 - - [10/Mar/2023:13:55:36 +0000] "GET /index.html HTTP/1.1" 200 1543
203.0.113.45 - - [10/Mar/2023:13:56:01 +0000] "GET /images/logo.png HTTP/1.1" 200 5289
```

任务要求（必做）：

- 程序能读取名为 `access.log` 的文件；
- 计算总请求数（即总行数）；
- 统计状态码大于等于 400 的请求数，并计算其占总请求数的百分比。

任务要求（选做，加分）：

- 找出访问次数最多的前 3 个 IP 地址及其访问次数；
- 找出访问量最大的小时（0-23）及该小时的总请求数。

输入格式：

- 一个名为 `access.log` 的文本文件，位于附件中。你也可以自行补充数据用于测试。

输出示例（含选做任务输出，仅供格式参考）：

--- 网站日志分析报告 ---

[总览]

总请求数：50

[高频IP Top 3]

1. 192.168.1.1 (12 次)
2. 203.0.113.45 (9 次)
3. 198.51.100.12 (7 次)

[流量高峰时段]

- 高峰时段：14点 (即 14:00 - 14:59)
- 请求总数：9 次

[错误请求分析]

- 错误请求 (状态码 ≥ 400): 8 次
- 错误率：16.00%

3. 提交内容

- 简答题提交一份由 Markdown 导出的 PDF 文件。
- 编程题提交一份代码文件。



二、网络和 HTTP

1. 简答题

1. HTTP 和 HTTPS 是什么？HTTP 请求和响应分别由哪些部分组成？
2. GET 与 POST 有什么区别？分别在什么场景下使用？
3. 为什么 API 返回数据普遍使用 JSON，而不是 XML？
4. 简述 Cookie、Session、Token 的使用场景。
5. 什么是 RESTful API？

2. API 设计题：用户管理系统

题目背景：

- 你正在开发一个"用户管理系统"，能够完成基本的 CRUD 操作。请先完成它的 RESTful API 设计。

核心实体

- 用户：唯一用户名，含密码。

核心功能

1. 用户注册；
2. 用户登录；
3. 查看当前登录用户的信息；
4. 查看用户列表；
5. 修改用户名；
6. 修改密码；
7. 注销账号。

要求

- 每个 API 必须包含：请求方法、URL、请求参数/请求体结构、状态码、响应体结构（不需要则写"无"）；
- URL 结构清晰、直观；
- 请求体、响应体均使用 JSON；
- 状态码使用规范；
- 提交为 Markdown 文档，格式参考下方示例。

示例 (Markdown 格式)

```
## 用户注册

- **请求方法**：`POST`
- **URL**：`/auth/register`
- **请求体**：
  {
    "username": "newuser",
    "password": "a_strong_password"
```

```
}

- **状态码和响应体**:
- `201 Created`: "用户创建成功"
  {
    "id": "user-uuid-12345",
    "username": "newuser"
  }
- `400 Bad Request`: "用户名已存在"
  {
    "message": "user already exists"
  }
```

3. 提交内容

- 简答题提交一份由 Markdown 导出的 PDF 文件。
- 设计题提交一份由 Markdown 导出的 PDF 文件。



三、关系型数据库与 SQL

1. 简答题

1. 主键、外键是什么？它们的作用和区别？
2. 什么是范式？1NF、2NF、3NF 分别要求什么？
3. 什么是事务？ACID 分别是什么含义？
4. 简述索引的作用。

2. 编程题：练习使用 SQL 语言

DDL：创建数据表

1. 根据附件 表设计与测试数据.pdf 中的规范，创建三张表（注意主键、外键、非空约束）。

DML：数据操作

1. 将附件中的测试数据插入到三张表中；
2. 将课程"数据库系统"（C001）的学分更新为 4.0；
3. 删除学生"刘洋"（S106）的所有选课成绩记录。

DQL：数据查询

1. 查询"计算机学院"所有学生的姓名和学号；
2. 查询姓名中含有"小"字的学生姓名和学号；
3. 查询每门课程的名称和平均分，按平均分从高到低排列；
4. 查询每门课程的最高分和最低分，显示课程名；
5. 查询所有不及格（成绩 < 60）学生的姓名和课程名；
6. 查询选课数量超过 2 门的学生姓名，按选课数量从高到低排列；
7. 查询每名学生的总学分（成绩 ≥ 60 才算通过，没有成绩的学生可不出现）；
8. 查询平均分最高的学生姓名及其平均分；
9. 查询没有选任何课程的学生姓名。

3. 提交内容

- 简答题提交一份由 Markdown 导出的 PDF 文件。
- 编程题提交一份 .sql 文件。

Part 2: 项目实战

四、用户管理系统

1. 简答题

1. 请描述一次"用户注册"的完整流程：从客户端发起 HTTP 请求，到数据写入数据库并返回响应，中间经历了哪些环节？
2. 为什么用户数据要持久化到数据库，而不是只存放在程序内存中？

2. 编程题

根据先前设计好的 API，开发一个小型用户管理系统。

使用的数据库可自选，推荐 MySQL、H2、SQLite

核心功能（必做）

1. 用户注册（密码允许明文存储）；
2. 用户登录，返回一个随机会话标识 sessionId（随机生成）；
3. 查看当前登录用户的信息（调用接口时在请求头携带 sessionId）。

登录态说明：登录成功后，服务端保存 sessionId 与 userId 的对应关系；需要登录态的接口从请求头读取 sessionId 并校验，查不到或为空时返回 401。

核心功能（选做，加分）

1. 查看用户列表；
2. 修改用户名；
3. 修改密码；
4. 注销账号。

推荐实现路线：

1. 创建用户表 —— 验收：能在数据库中成功创建 users 表；
2. 建立数据库连接 —— 验收：程序能连接数据库并读取数据；
3. 实现"用户注册"接口 —— 验收：Postman 中访问注册接口返回 201，且数据库中新增一行用户信息；
4. 实现"用户登录"与"查看当前登录用户" —— 验收：登录成功返回 sessionId；携带 sessionId 能查到当前用户信息；不带或带错则返回 401；
5. 实现其余接口（用户列表、修改用户名、修改密码、注销账号）；
6. 处理边界情况与状态码规范 —— 验收：重复注册返回 409（或 400），操作不存在的用户返回 404，未登录返回 401。

3. 提交内容

- 将项目上传至 **GitHub**，提交一份 `.txt` 文件记录你的 GitHub 公共仓库网址。
- 若无法连接 GitHub，提交项目文件夹。

- 请使用 Postman（或 Apifox、Bruno 等同类 API 调试工具）测试接口，并附上测试结果截图。



五、挑战题

本题为选做加分题，未完成不影响前面的提交结果；若挑战题只完成部分功能，也可以提交。

1. 简答题

1. 为什么用户密码不能明文存储在数据库中？简述「哈希 (Hash)」和「盐值 (Salt)」的作用。
2. 什么是「统一异常处理」？相比在每个接口里单独写 `try-catch`，它有什么优势？
3. JWT (JSON Web Token) 由哪三部分组成？服务端如何验证 Token 未被篡改？若 Token 过期，接口应如何响应？

2. 扩展用户管理系统

在第四大题的用户管理系统基础上，继续进行扩展。

添加功能

1. JWT 认证

1. 登录成功后返回 JWT；
2. 查看当前用户、修改用户名、修改密码、用户注销等接口，需携带 `Authorization: Bearer <token>` 并校验身份；
3. 未携带、伪造或过期的 Token 均返回 401。

2. 密码安全存储

1. 数据库中禁止存储明文密码；
2. 注册和修改密码时，使用密码哈希算法对密码进行加密存储；
3. 登录时，通过哈希比对验证密码。

3. 统一的异常处理

1. 引入全局异常处理机制，捕获业务异常与系统异常
2. 对各项无效操作返回对应的状态码
3. 错误统一返回 `{"message": "..."}` 结构

4. 日志记录

1. 使用日志框架记录关键操作：
 - 用户登录成功/失败；
 - 用户注册成功；
 - 修改密码成功；
 - 日志需包含时间戳、操作类型与相关用户标识。

3. 提交内容

- 将项目继续上传至第四大题用户管理系统的仓库，提交一份 `.txt` 文件记录 GitHub 公共仓库网址；
- 若无法连接 GitHub，提交项目文件夹；
- 请使用 Postman（或 Apifox、Bruno 等同类 API 调试工具）测试接口，并附上测试结果截图。